SCALE Grant Proposal: Hewn

> Free, open-source bookkeeping and payroll for small businesses, private by design, built on Aleo. Live on testnet at www.hewnaccounting.com.

Applicant: Joshua Daniel Day · **Request:** $60,000 over 12 months · **Licence:** AGPL-3.0

Code: github.com/PNWBNW/hewn (app) · github.com/PNWBNW/pnw_mvp_v2 (Leo programs)

Since we last applied

We applied before as **Proven National Workers (PNW)**, a payroll portal with one private payroll run on testnet. The committee told us it wasn't built out far enough. That was fair. Since then:

Then (PNW)	Now (Hewn, October 2026)
A payroll portal: pay a worker, mint a receipt	A full small-business app: books, invoices, bills, payroll, timecards, scheduling, reports, sales tax, 1099s, hiring, messaging
One private payroll run, from a demo flow	Pay runs built into the books: each worker paid privately in USDCx from the business's own Shield wallet, the paystub sent as a sealed record
Data shared through IPFS and a pinning service	**No server holds anything.** Every message, offer, timecard and paystub is a private Aleo record; the chain is the mailbox
Proprietary licence	**AGPL-3.0**, app and Leo programs
11 programs	16 programs live on testnet and more staged, each with its own README; the new ones with must-fail tests
A few hundred tests	~1,100 automated tests, a written threat model, a manual test guide for outside developers

This grant isn't to build Hewn: Hewn is built. It's to take it from a working testnet product to mainnet, with a legal entity behind it and one or two more people building it.

Project Overview

The problem

A small business today hands its most sensitive data to two or three companies: books to QuickBooks or Xero, payroll to Gusto or ADP, workers' Social Security numbers, wages, bank details and W-4s to all of them. That data sits in centralised databases that get breached, cost more every year, and lock the business in. Workers have no say in where their pay and identity go.

In Web3 it's worse in the other direction: on-chain payroll tools exist, but they pay in public. Anyone can read every contributor's pay.

Small businesses need software that does what QuickBooks and Gusto do, without anyone in the middle holding their data, and without putting anyone's pay in public.

What Hewn is

Hewn is a free, open-source bookkeeping and payroll app that works the way bookkeepers already expect (chart of accounts, double-entry books, invoices, bills, payroll, reports) with one difference: **nothing about the business or its people is held by anyone but them.**

- **The books live in an encrypted vault in the browser.** AES-256-GCM, unlocked by the user's username and passphrase. No Hewn server, no database, no account to breach.
- **The wallet is the bank feed.** Hewn reads the business's Shield wallet history and asks what each payment was for, instead of making anyone type transactions in.
- **Payroll is private by construction.** Hewn computes gross pay, overtime, on-call pay and federal withholding on the device, then pays each worker with a private USDCx transfer signed in the business's own Shield wallet. Hewn never holds the money.
- **Everything between two people is a sealed record on Aleo.** Offers, acceptances, timecards, schedules, paystubs, messages and team announcements travel as private records, sealed twice: to the recipient's account, and inside, with a key only the two parties hold. The chain is the mailbox; there is no message server to subpoena or breach.
- **Hiring in the open, applying in private.** A public job board on chain (postings are meant to be read), with applications and resumes sent as sealed records only the business can open.
- **Names, not addresses.** Every business and worker registers a `name.hewn`, and finds others by name.

Two sides in one app: a **business** keeps its books and runs its team; a **worker** clocks in, confirms their schedule, accepts offers and keeps every paystub. A **customer** side (hire a business, pay its invoices in USDCx, review it only after paying) is designed and next.

Why it needs Aleo, and why privacy matters here

Payroll is the clearest case for zero-knowledge there is: everyone understands "your pay shouldn't be public." A worker's wage, withholding, W-4 and bank details are among the most protected personal data there is, and a business's payroll reveals its headcount, its costs and who works for whom.

Hewn uses Aleo's primitives directly:

Aleo primitive	What Hewn does with it
Private records	Every letter, offer, agreement, timecard and paystub is a record only its owner can decrypt. No public counterpart.
Private USDCx transfers with Sealance exclusion proofs	Wages move privately, wallet to wallet, with the compliance proof built in the browser.
Public mappings with only hashes and heights	Agreement status, the job board and name ownership are public where they must be, and never hold a wage, an address or a name in plain text. A CI check rejects cumulative counters (they leak wages by difference).
ZK proofs in the user's wallet	Every write is proven and signed in Shield, so the user sees exactly what they sign. Nothing is signed on a server.
Key exchange over Aleo records	A business and each worker agree a fresh conversation key by P-256 key exchange, its public halves carried inside Aleo records and the key itself never sent, so even a view key holder can't read their messages after the handshake. Both sides must agree before a conversation is backed up.

Impact on the Aleo ecosystem

- **Recurring, real-world transactions.** A business that runs payroll on Hewn creates transactions every week, not once: a USDCx transfer and a paystub record per worker per pay run, plus weekly timecards, schedules and messages, each a record on Aleo. A 10-person team paid every two weeks generates on the order of 100–200 transactions a month. (An estimate from how the app works; we'll measure it on mainnet.)
- **USDCx with a reason to exist.** Every business that pays on Hewn holds and moves USDCx. With the customer side, its customers do too.
- **New users who never thought of themselves as crypto users.** Hewn is aimed at plumbers, contractors and shops, not traders. The word "blockchain" barely appears in the app; the wallet is just how money moves.
- **An open-source reference app.** AGPL: every Leo program, the wallet integration and the privacy rules are public for other teams to learn from or fork.

Team Background

Joshua Daniel Day: founder and developer, building with Claude Code

- Designed and built Hewn and its 16 testnet programs, on nights and weekends alongside a full-time job: learning Aleo from late 2023, building PNW from February 2025, Hewn from October 2026.
- Ran an end-to-end private payroll on Aleo testnet from a browser, during the Aleo Buildathon (April 10, 2026; transactions below).
- Builds with Claude Code, Anthropic's AI coding agent, as a development partner under strict, written project rules: a privacy floor, one execution boundary to the chain, must-fail tests for every program, and a dated log of every change. The rules and the whole history are public in the repository (`CLAUDE.md`, `BUILD_PLAN.md`, `PROGRESS.md`). That is how one person with a day job built this much, carefully.
- GitHub: PNWBNW · LinkedIn: *[add]*

With thanks to Kufre-abasi Bassey, who has tested Hewn and helped shape it from the earliest idea-board stages.

What the grant adds: one or two part-time developers (Leo and TypeScript, and testing), and an LLC to own the project, hold its funds and sign with partners.

Market Potential & Demand

Who it's for

Segment	Size	What they need that Hewn does
US small businesses with employees	About 33 million US small businesses, about 6 million with employees (SBA Office of Advocacy)	Books and payroll without handing workers' data to a vendor; no monthly subscription
Trades and field-service businesses (contractors, plumbers, electricians, crews)	A large share of the above	Timecards, overtime, on-call pay and callouts, scheduling, job costing, paying subcontractors: all built in
Crypto-native teams and DAOs	Thousands of on-chain organisations	Paying contributors without publishing their pay
Workers	Every person paid through a Hewn business	Their own paystubs, schedule and job history, kept by them, not by their employer's vendor

The pain

- **Cost that keeps rising.** QuickBooks Online plans run from about $35 to over $250 a month, and payroll add-ons charge per employee on top. Hewn's bookkeeping is free; the plan is a small flat fee per paycheck paid on chain (cents, capped per business), never a percentage of wages.
- **Data held by others.** Payroll vendors hold Social Security numbers, wages and bank details for tens of millions of workers, which makes them a standing target for breaches. Hewn holds none.
- **Public pay on chain.** Existing Web3 payroll pays in the open. Hewn's pay is private by default and can't be made public.

Signals

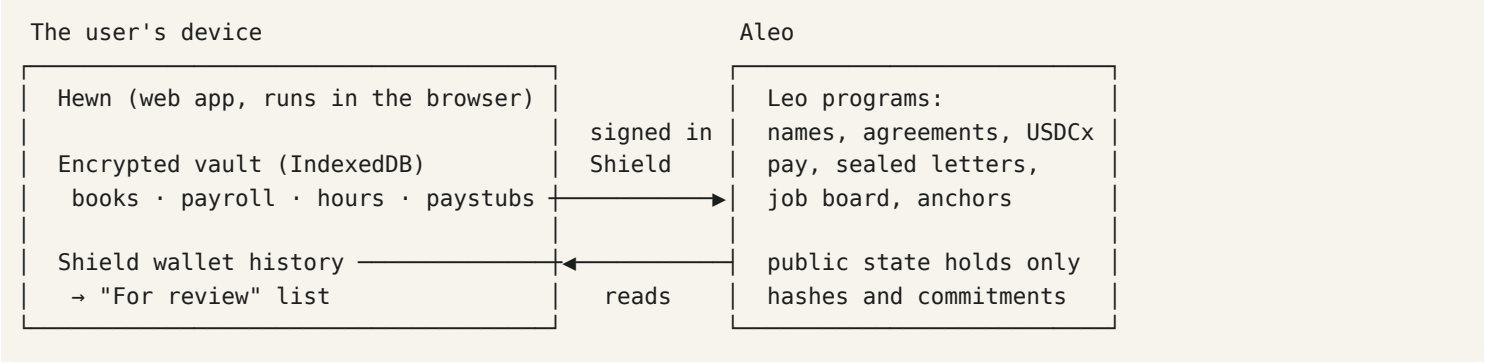
- Payroll and accounting software is a large, steadily growing market, and small businesses keep moving to cloud tools.
- Privacy law keeps expanding (GDPR, CCPA and many new US state privacy laws), raising every employer's liability for the worker data it holds. Hewn's answer is to hold none.
- Stablecoin payments are moving into the mainstream, and USDCx gives Aleo a private one.
- Hewn's public **traction map** (www.hewnaccounting.com/traction) counts visits by country with no personal data. *[Add current numbers before submitting.]*

How Hewn compares

	Private wages	Private identities	No vendor holds the data	Books and payroll together	Open source
QuickBooks + Gusto / ADP	No	No	No	With add-ons	No
Public on-chain payroll tools	No	No	Partly	No	Some
Hewn	**Yes**	**Yes**	**Yes**	**Yes**	**Yes (AGPL)**

System Design & Specification

Architecture



1. The app (Next.js, about 69,000 lines of TypeScript, 17,000 of them tests). Runs entirely in the browser, as a website and as an installable app. No backend holds user data; the only server-side code serves the page and one anonymous traction count.

2. The vault (`packages/vault`). A random AES-256-GCM data key encrypts the books; the unlock secret (PBKDF2-SHA256, 600,000 rounds) only wraps it, so recovery keys and multi-member access later just add wrapped copies.

3. The ledger (`packages/ledger`). Double-entry core with a negative test for every rule. Payroll, sales and expenses post into it. Reports (P&L, balance sheet, cash flow, trial balance, aging, project profitability, sales tax) compute on the device.

4. The chain boundary (`apps/business/src/chain`, `packages/wallet`, `pnw-adapter`). The only code that touches Aleo, loaded only on pages that need it. A test fails the build if the ledger, vault or payroll ever import it. Every write goes through Shield's executor.

5. Leo programs (pnw_mvp_v2, Leo 4, about 5,300 lines live):

Program	Role	Status
`pnw_name_registry_v2`, `pnw_name_registrar_v5`	Who owns which `name.hewn`; registration	Live
`pnw_worker_profiles_v2`, `employer_profiles_v2`	Private profiles behind a name	Live
`employer_agreement_v4`	Offers and agreements: offer, accept, pause, resume, terminate	Live
`employer_agreement_v5`	The same, with the key-exchange halves in the records and no public ownership check	Staged, deploying
`hewn_letters_v1`, `hewn_letters_v2`	Sealed letters: one, many, and long pages	Live
`hewn_jobs_v1`	The public job board, with USDCx posting and feature fees	Live
`payroll_core_v2`, `paystub_receipts`, `payroll_audit_log`, `pnw_router_v4`	The original one-proof payroll, receipts and audit anchors	Live, kept for the audit design
`payroll_nfts_v2`, `credential_nft_v3`, `audit_nft`, `employer_license_registry`	Anchors, credentials and audit consent	Live, waiting on the audit design (`credential_nft_v4` staged)

6. Documentation. Every design is written before it's built, and kept current: threat model, sealed letters, key exchange, job board, audit design, customers, build plan, and a dated progress log of every change.

What's next, and what it depends on

Component	Depends on	Months
Agreements v5 live: every business–worker conversation key-exchanged from the first message	Deploy (in progress)	1
A real-business pilot on testnet, with outside developers running the manual test guide	—	1–3
Customers: service agreements with a clear end (complete, warranty, closed); invoices paid in USDCx straight to the business, with receipts and a review ticket in one transaction; reviews only after paying	Two new Leo programs; a probe of Shield's prover on a private USDCx call inside another program	2–6
A directory of businesses taking clients, and requests only businesses in the chosen field can read	One new Leo program	4–7
`.hewn` as a top-level name in the Aleo Name Service, so names show in every ANS-aware wallet and explorer	A conversation with the ANS team; our registrar program	5–8
Independent security review of the programs that move money	Programs frozen	7–9
Mainnet	USDCx on mainnet; the review's fixes	9–12

Known technical challenges:

- **Shield's in-browser prover** handles one program per proof best, so payroll runs as one transaction per worker rather than one batch proof. The customer payment design needs a probe before it's built, with a fallback if it fails.
- **Testnet availability.** Deploying programs costs testnet credits, and the faucet and parts of testnet were down at the time of writing, which held up our latest deploy.
- **Phone use without the extension** depends on Shield's relay, which is built but not yet tried on a real phone.

Funding & Milestones

Request: $60,000 over 12 months. We're asking for validation as much as money: enough to keep building, bring on help, and set up a proper company to take Hewn to mainnet.

Budget

Line	What it pays for	Amount
Founder development time	Twelve months of building, moving from nights and weekends toward more hours	$24,000
One or two part-time developers	Leo and TypeScript work on the customer side and mainnet, and testing on real devices	$18,000
LLC and legal	Forming the company, an operating agreement, terms of service and privacy policy, and a lawyer's review of the payments model (Hewn never holds funds)	$6,000
Security review	An independent review of the programs that move money, before mainnet	$9,000
Infrastructure and mainnet deploy	Hosting, domain, and the network fees to deploy the programs on mainnet	$3,000
	Total	**$60,000**

The security line covers a focused review of the programs that move money. A full audit of every program costs more; we'd welcome an introduction to Aleo's audit partners for it.

Milestones

Milestone 1, Months 1–3: Foundations ($15,000)

- Agreements v5 live on testnet; business–worker conversations key-exchanged from the start.
- A complete pay run from Hewn on testnet (timecards to paystubs), transaction hashes published.
- LLC formed.
- A pilot with at least one real small business keeping its books and running its team on Hewn (testnet money for pay).
- Outside developers working through the manual test guide, results on GitHub.
- **Done when:** v5 and pay-run transactions on testnet; the LLC's filing; pilot feedback logged.

Milestone 2, Months 3–6: Customers ($15,000)

- Service agreements and paying invoices in USDCx through them (receipt and review ticket in one transaction), live on testnet, with must-fail tests.
- Customer accounts and the business directory.
- **Done when:** a customer hires a business, pays two invoices in USDCx and leaves a review that goes public at closure, all on testnet.

Milestone 3, Months 6–9: Mainnet readiness ($15,000)

- `.hewn` names under the Aleo Name Service (or our own registry if ANS declines), on testnet.
- The independent security review, and its fixes.
- **Done when:** a `.hewn` name resolves in an ANS-aware explorer; the review report has no open critical findings.

Milestone 4, Months 9–12: Mainnet ($15,000)

- Programs deployed on mainnet; the app switched over.
- First businesses paying real people in USDCx.
- Guides for businesses, workers and developers.
- **Done when:** 10 businesses and 50 workers active on mainnet.

Evaluation Criteria

1. Impact

- **Commercialization and adoption:** a product for businesses that have never used a wallet, with a reason to come back every week (payroll, timecards, schedules).
- **Payments:** payroll, and next customer invoices, settle in private USDCx, wallet to wallet. This is a payments application first.
- **Identity:** every account is a registered `name.hewn`; on mainnet, a top-level name in the Aleo Name Service.
- **Network activity:** recurring transactions per worker per pay period, plus every message and document between people.

2. Readiness

- **Easy to medium.** The app works on testnet today with Aleo's existing tools: Leo 4, Shield and Provable's wallet adapters, USDCx with Sealance proofs, the Provable explorer.
- **Dependencies:** USDCx on mainnet; the ANS team for the `.hewn` name; Shield's prover for the customer payment design (with a fallback).

3. Collaborative value

- **Shield:** Hewn is built on Shield alone (extension, Shield's browser, and the relay for phone browsers), and reports what it finds (fee modes, multi-program proofs).
- **USDCx and Sealance:** a real consumer of private stablecoin transfers, with exclusion proofs built in the browser.
- **Aleo Name Service:** `.hewn` as a new top-level name, bringing every Hewn business and worker into ANS.
- **zPass, later:** verified employment history (a worker proving they held a job, or earned over an amount) without showing the paystubs.
- **Developers:** an AGPL reference app with a threat model, must-fail tests for every program, and a progress log that explains every decision.

Supporting Materials

Live app (testnet): www.hewnaccounting.com · public job board: /board · traction map: /traction

The first private payroll on Aleo testnet (April 10, 2026):

Step	Program	Transaction
Verify agreement	`employer_agreement_v4`	`at1mydsktdsr8pk7d4utzrp6n2rvtgkkpavthukyt4kpdadgyx5lg8sgljea3`
Transfer USDCx	`test_usdcx_stablecoin`	`at1yphn8n9zejqnnsktuev7rl9vkv8styq00rjpffa0h7rxnccssyyqdk9ltw`
Mint receipts	`paystub_receipts`	`at1w86wy80c9sgv0e2ukwlzja4r4km0vkld2tna586t9447q6pjvvrqhuvnw4`
Anchor event	`payroll_audit_log`	`at1jp6mertn92hpn79uak8vdy9t4ha2t0f4fwq877uy6rmjl20g0syqdzygp3`
Mint cycle NFT	`payroll_nfts_v2`	`at1d8ht598hqqjgmqfxjwvt0cf47aqafgynzjhazhtreze6j22hzcrq5992r5`

Programs deployed since (examples):

- `hewn_letters_v1`, the first sealed letters (October 16, 2026): deploy `at1pxclare6clej0feuzcncngmwvma44g0hg2ngsh763gltk0rz6qpqgtfqnt`; a letter sent from a phone in Shield, `at1gl7jhc6utv47le0e20v0sywqhtrrludlt9mt7w4y4tdluzr3vu8qeddmsn`.
- `hewn_letters_v2`: `at177xpwtt9kdlffahrrnzdwv335g9lys63auye7zw3gtju09ghxu8q9jlauk`.
- `hewn_jobs_v1`, the job board: `at1354m4j9wxc9st3ukwcl2082s6429ddgx27kfekz500rpzcne4qqse80q4n`.

Contact: grants correspondence to *[your email]*.